

Boing Network — JSON-RPC API Specification

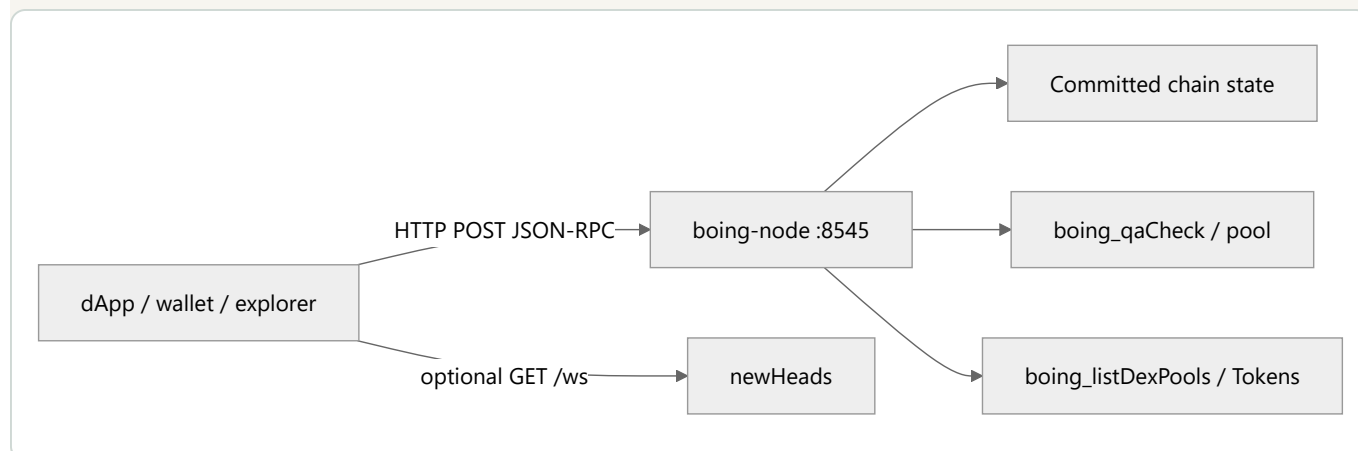
- 👤 **Everyday users:** you do not call this API directly. The [wallet](#) and [explorer](#) do it for you.
- 🔧 **Developers:** this is the contract. Prefer `boing-sdk createClient(url)` over hand-rolled `fetch`. Probe with `boing_getRpcMethodCatalog` / `probeBoingRpcCapabilities`.
- 🛠️ **Operators:** CORS, batch limits, `/live` `/ready`, and `BOING_*` env for `boing_getNetworkInfo.end_user`.

Version: 0.1

Transport: HTTP POST

Encoding: JSON-RPC 2.0

References: [RUNBOOK.md](#) · [BOING-RPC-ERROR-CODES-FOR-DAPPS.md](#)



Public testnet: <https://testnet-rpc.boing.network/> . Local default: <http://127.0.0.1:8545/> .

If `boing_chainHeight` works but newer methods return `-32601`, the process is an **older binary** or a gateway allowlist — upgrading VibeMiner on your laptop does **not** change the public explorer. See [THREE-CODEBASE-ALIGNMENT.md](#) §2.1.

Overview

Boing nodes expose a JSON-RPC HTTP interface for submitting transactions, querying chain state, and simulation. Rate limiting applies per `RateLimitConfig` (see [SECURITY-STANDARDS.md](#)). When enabled, **one HTTP POST / consumes one token** even if the body is a **JSON-RPC batch** (array of requests). Responses

use HTTP **429** with header **Retry-After: 1** (second) and a JSON-RPC error body (**-32016**) so clients can backoff without parsing the body. The same JSON body may include a sibling **boing_http** object (**code** , **message** , **request_id**) for log correlation.

Diagnosing **-32601** (method not found)

If **boing_chainHeight** works but newer methods (e.g. **boing_getSyncState** , **boing_getNetworkInfo** , **boing_getLogs** , **boing_getTransactionReceipt**) return **-32601 Method not found**, the HTTP endpoint is **not** running the same RPC surface as **current boing-node** in this repository: the process is usually an **older binary** or traffic is going through a **gateway that only whitelists some method names**. Rebuild and restart the node from this repo (`cargo build -p boing-node --release` , see [RUNBOOK.md](#)), or point the client at the node's JSON-RPC port with no filtering.

Use **boing-sdk probeBoingRpcCapabilities** / `npm run probe-rpc` (from **boing-sdk** after `npm run build` , or from the repo root) — the printed JSON includes **clientVersion** / **supportedMethodCount** when **boing_clientVersion** / **boing_rpcSupportedMethods** exist; the **diagnosis** field explains gaps when **explainBoingRpcProbeGaps** applies. For a single pass that also checks **GET /openapi.json** , **/.well-known/boing-rpc** , and **/live.json** , run `npm run boing:smoke` from the repo root (after `npm run build` in **boing-sdk**) or call **doctorBoingRpcEnvironment** in code.

Base URL

```
http://<host>:<rpc_port>/
```

Default RPC port: **8545** .

The same HTTP listener serves **GET /live** , **GET /ready** , **GET /ws** , **GET /openapi.json** , **GET /.well-known/boing-rpc** , and **POST /** (JSON-RPC). **GET /** (root) returns **405 Method Not Allowed** with **Allow: GET, HEAD, POST, OPTIONS** and a short plain-text hint so accidental browser or `curl` visits fail clearly. **HEAD /** returns the same **405** and **Allow** with an empty body. **OPTIONS /** advertises the same allowed methods. The in-tree handler returns **204** with no body; the **CORS** middleware may respond to browser preflights with **200** before the route runs—either way, integrators can rely on **Allow** for discovery.

JSON probes: **GET /live.json** (or **GET /live** with **Accept: application/json**) returns **{ "ok": true }** . **GET /ready.json** (or **Accept: application/json** on **/ready**) returns **{ "ready": true }** , or **503** with JSON **{ "ready": false, "reason": "peers_below_min", "peers", "min_peers" }** when **BOING_RPC_READY_MIN_PEERS** is set and peers are below the threshold (**Retry-After: 5** on **503**). **HEAD /live** and **HEAD /ready** return status only.

HTTP OpenAPI: **GET /openapi.json** returns the same OpenAPI 3.1 object as **boing_getRpcOpenApi** . **GET /.well-known/boing-rpc** returns a small JSON document with **schema_version** and path hints (no JSON-RPC round-trip required).

Request correlation: Clients may send `X-Request-Id` on any request; the node echoes it on the response. If omitted, the node assigns a UUID. Browsers reading this header from cross-origin responses need `CORS Access-Control-Expose-Headers` (the node exposes `x-request-id`, `retry-after`, and `x-boing-rpc-version`).

Response headers: Successful JSON-RPC responses include `X-Boing-RPC-version` (e.g. `boing-node/0.1.0`) so proxies and clients can detect binary skew without an extra `boing_clientVersion` call. The same header is set on `/live` and `/ready`.

Request body limit: `POST` / JSON bodies default to **8 MiB** max (large signed-transaction hex). Set `BOING_RPC_MAX_BODY_MB` (integer ≥ 1) to override. Oversized bodies receive HTTP **413** with `Content-Type: application/json` and body `{ "code": "payload_too_large", "message", "request_id" }` when `x-request-id` is available on the response.

JSON-RPC batch: The body may be a **JSON array** of request objects. The HTTP response is a **JSON array** of response objects in the same order. Requests **without an id** are treated as **notifications** and produce **no entry** in the batch response. If **every** entry in the batch is a notification, the server returns **HTTP 204** with **no body** (per JSON-RPC 2.0). A **single** notification (non-batch) also returns **204**. An empty batch array `[]` returns **200** with body `[]`. Max batch length defaults to **32** and is capped at **256**; set `BOING_RPC_MAX_BATCH` to override (invalid values fall back to the default).

Browser CORS

`boing-node` sends **CORS** headers for a built-in allowlist (Boing web apps and common local dev ports). To allow **additional** browser origins without recompiling, set environment variable `BOING_RPC_CORS_EXTRA_ORIGINS` to a comma-separated list (e.g. `http://localhost:9999,https://preview.example.com`). Invalid entries are skipped with a warning in node logs.

Chain tip, committed height, and finality

Boing uses a **HotStuff-style BFT** consensus layer (`boing-consensus`): a block is appended to this node's `ChainState` only after it is **committed** through that path. There is **no separate JSON-RPC "unsafe head"** ahead of commit in the current node implementation.

Term	Meaning in this codebase
<code>boing_chainHeight</code> / <code>head_height</code> in <code>boing_getSyncState</code>	Height of the latest committed block stored on this node (the chain tip).
Finalized (protocol sense)	Under standard BFT assumptions (honest quorum, etc.), a committed block is not reverted by consensus.
<code>finalized_height</code> in <code>boing_getSyncState</code>	Today equal to <code>head_height</code> , because the node only exposes committed blocks. Reserved so a future release can report lag (e.g. optimistic execution vs commit) without breaking clients.
Syncing / lagging peers	A node that is still catching up may report a lower height than the rest of the network until it imports commits. Clients should not treat “height stopped increasing” as finality across the whole network without comparing multiple sources.

Wallets and observers that need a single number for “how deep is my tx” can use `boing_chainHeight` or `boing_getSyncState`; until multiple heights diverge in a future version, they are equivalent for “committed tip.”

dApp network discovery

`boing_getNetworkInfo` (params `[]`) returns a single JSON object for wallets and dApps: committed tip fields (same as `boing_getSyncState`), target block time, `client_version`, optional `chain_id` / `chain_name` from environment (see below), consensus summary (`validator_count`, `model`), `native_currency`, `chain_native` (chain-wide sums over the committed account table), `developer` (doc URLs, npm package id, WebSocket `/ws` handshake, and API discovery method names), and an explicit `rpc.not_available` list.

`chain_native: account_count`, `total_balance`, `total_stake`, and `total_native_held` are **u128 decimal strings** (same style as `boing_getAccount`). They are the **sum** of per-account `balance` / `stake` in this node’s committed state — **not** “circulating supply”, “total minted”, or treasury definitions. `as_of_height` is the committed tip height those sums match.

Important: Public JSON-RPC still does **not** expose **staking APY** or protocol-defined **network-wide supply** (mint/burn/treasury). The node lists remaining gaps under `rpc.not_available` (today `staking_apy`) and `rpc.not_available_note`. **Per-account** balance and stake remain available via `boing_getAccount` (and `boing_getBalance` for balance only).

Operator environment (optional):

Variable	Effect on <code>boing_getNetworkInfo</code>
<code>BOING_CHAIN_ID</code>	If set to a decimal integer (e.g. <code>6913</code> for public testnet), included as JSON number <code>chain_id</code> . If unset or invalid, <code>chain_id</code> is <code>null</code> — clients must use their configured chain id (e.g. from app env).
<code>BOING_CHAIN_NAME</code>	Optional human-readable <code>chain_name</code> (e.g. <code>Boing Testnet</code>). Omitted when unset.
<code>BOING_CHAIN_DISPLAY_NAME</code>	Optional wallet-facing display string in <code>end_user.chain_display_name</code> (does not replace <code>chain_name</code>).
<code>BOING_EXPLORER_URL</code>	Optional block explorer base URL in <code>end_user.explorer_url</code> .
<code>BOING_FAUCET_URL</code>	Optional faucet URL in <code>end_user.faucet_url</code> .
<code>BOING_CANONICAL_NATIVE_CP_POOL</code>	Optional 32-byte <code>AccountId</code> hex (with or without <code>0x</code>) in <code>end_user.canonical_native_cp_pool</code> — public RPC operators publish the canonical native constant-product pool so dApps need not hardcode it (OPS-CANONICAL-TESTNET-NATIVE-AMM-POOL.md). Invalid values are ignored with a warning in node logs.
<code>BOING_CANONICAL_NATIVE_DEX_FACTORY</code>	Optional pair-directory contract <code>AccountId</code> in <code>end_user.canonical_native_dex_factory</code> for default <code>boing_getContractStorage</code> / <code>boing_getLogs</code> wiring.
<code>BOING_CANONICAL_NATIVE_DEX_MULTIHOP_SWAP_ROUTER</code>	Optional multihop (2–6 pool) swap router <code>AccountId</code> in <code>end_user.canonical_native_dex_multihop_swap_router</code> .
<code>BOING_CANONICAL_NATIVE_DEX_LEDGER_ROUTER_V2</code>	Optional ledger forwarder <code>AccountId</code> (160-byte inner calldata) in <code>end_user.canonical_native_dex_ledger_router_v2</code> .
<code>BOING_CANONICAL_NATIVE_DEX_LEDGER_ROUTER_V3</code>	Optional ledger forwarder <code>AccountId</code> (192-byte inner calldata) in <code>end_user.canonical_native_dex_ledger_router_v3</code> .
<code>BOING_CANONICAL_NATIVE_AMM_LP_VAULT</code>	Optional LP vault <code>AccountId</code> in <code>end_user.canonical_native_amm_lp_vault</code> .
<code>BOING_CANONICAL_NATIVE_LP_SHARE_TOKEN</code>	Optional LP share token <code>AccountId</code> in <code>end_user.canonical_native_lp_share_token</code> .

`rpc_surface` on `boing_getNetworkInfo`: mirrors `boing_health.rpc_surface` (batch cap, body limit, `boing_getLogs` bounds, WS cap, rate limit, ready peer gate).

Request Format

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "boing_chainHeight",
  "params": []
}
```

Response Format

Success:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": 42
}
```

Error:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid params: expected [hex_signed_tx]"
  }
}
```

Methods

Method index (`boing_rpcSupportedMethods`)

Current `boing-node` implements these JSON-RPC methods (same set returned by `boing_rpcSupportedMethods` , sorted alphabetically):

```
boing_chainHeight, boing_clientVersion, boing_faucetRequest, boing_getAccount,
boing_getAccountProof, boing_getBalance, boing_getBlockByHash, boing_getBlockByHeight,
boing_getContractStorage, boing_getDexToken, boing_getLogs, boing_getNetworkInfo,
boing_getQaRegistry, boing_getRpcMethodCatalog, boing_getRpcOpenApi, boing_getSyncState,
boing_getTransactionReceipt, boing_health, boing_listDexPools, boing_listDexTokens,
boing_listSlashRecords, boing_operatorApplyQaPolicy, boing_qaCheck, boing_qaPoolConfig,
boing_qaPoolList, boing_qaPoolVote, boing_registerDappMetrics, boing_resolveSlashAppeal,
boing_rpcSupportedMethods, boing_simulateContractCall, boing_simulateTransaction,
boing_submitIntent, boing_submitSlashAppeal, boing_submitTransaction,
boing_verifyAccountProof.
```

Implementation: `BOING_RPC_SUPPORTED_METHODS` in `crates/boing-node/src/rpc.rs` — keep this list, the router match arms, and this spec in sync when adding a method.

boing_submitTransaction

Submit a signed transaction to the mempool.

Field	Type	Description
Params	<code>[hex_signed_tx]</code>	Hex-encoded bincode-serialized SignedTransaction

Example:

```
{"jsonrpc": "2.0", "id": 1, "method": "boing_submitTransaction", "params": ["0x..."]}
```

P2P: When `--p2p-listen` is set and the tx is **admitted** to the mempool, the node also **gossips** the same `SignedTransaction` on libp2p topic `boing/transactions` so peers can verify and enqueue it locally ([TECHNICAL-SPECIFICATION.md](#) §12.3, [RUNBOOK.md](#) §8.1).

Result (success): `{ "tx_hash": "ok" }` — current `boing-node` uses the literal `"ok"` when the tx is accepted into the mempool (not a 32-byte transaction id). Track inclusion via `boing_getBlockByHeight` / receipts / account nonce, or compute the signable `tx_id` client-side from the signed payload if your tooling needs a stable hex.

QA pool (Unsure): If mempool QA returns Unsure **and** governance allows the pool to accept work (`qa_pool_config: non-zero max_pending_items` and either `administrators` or `dev_open_voting`), the node enqueues the deployment and responds with `-32051` and `data: { "tx_hash": "0x..." }`.

Governance-listed administrators vote via `boing_qaPoolVote`. Hard caps (`max_pending_items`, `max_pending_per_deployer`) prevent pool congestion; when full, `-32055` / `-32056` apply instead of enqueueing.

Operator RPC (optional): When the node process has environment variable `BOING_OPERATOR_RPC_TOKEN` set to a non-empty string, `boing_qaPoolVote` and `boing_operatorApplyQaPolicy` require HTTP header `X-Boing-Operator: <same token>`. If the variable is unset, behavior matches earlier releases (no header check). Use this on any RPC endpoint reachable from untrusted networks so pool votes cannot be triggered by spoofing an admin hex alone.

Does the pool need RPC to “run”? No. The node **owns** the pool: when QA returns Unsure and governance allows it, enqueueing happens inside normal transaction/mempool handling (`boing_submitTransaction` may return `-32051`). No operator client is required for items to enter the queue or for the node to age them out per config. JSON-RPC is how **operators inspect and change** the pool—** `boing_qaPoolList` **, `boing_qaPoolConfig` , `boing_qaPoolVote` , `boing_operatorApplyQaPolicy` . For routine governance work, the **Boing Network desktop hub** (QA operator view) calls those methods over HTTP, so a terminal or **boing CLI** is optional (CLI remains useful for scripts and file-based `boing qa apply`).

boing_listSlashRecords

Read-only view of the node’s in-memory `SlashRegistry` (equivocation and liveness slashes + appeals).

Field	Type	Description
Params	<code>[]</code>	No parameters

Result:

```
{
  "slashes": [
    {
      "id": 1,
      "validator": "0x...",
      "amount": "5000",
      "reason": "equivocation",
      "block_height": 7,
      "appeal_deadline": 1007,
      "reversed": false
    }
  ],
  "appeals": [
    {
      "id": 1,
      "slash_id": 1,
      "status": "pending",
      "evidence_len": 12
    }
  ],
  "persisted": false
}
```

Notes: When the node has a **data directory**, the registry is written to `slash_registry.json` and `persisted` is `true`. Appeal submit/resolve: `boing_submitSlashAppeal` / `boing_resolveSlashAppeal` (operator auth when `BOING_OPERATOR_RPC_TOKEN` is set). See [TECHNICAL-SPECIFICATION.md](#) §12.3.

boing_submitSlashAppeal

Submit an appeal against a recorded slash (within the appeal window).

Field	Type	Description
Params	<code>[slash_id, evidence_hex]</code> or <code>{ "slash_id", "evidence" }</code>	<code>slash_id</code> number; <code>evidence_hex</code> (optional <code>0x</code> , may be empty)
Auth	Operator	Same as <code>boing_qaPoolVote</code> when <code>BOING_OPERATOR_RPC_TOKEN</code> is set

Result: `{ "appeal_id": N, "slash_id": N }`

Errors: `-32060` with registry error message (not found, window closed, duplicate, ...).

boing_resolveSlashAppeal

Approve or reject a pending appeal. Approval restores burned stake from the fee burn sink.

Field	Type	Description
Params	<code>[appeal_id, approved]</code> or <code>{ "appeal_id", "approved" }</code>	<code>approved</code> bool (or string <code>true</code> / <code>approve</code> / <code>1</code>)
Auth	Operator	Same as <code>boing_qaPoolVote</code> when <code>BOING_OPERATOR_RPC_TOKEN</code> is set

Result: `{ "appeal_id": N, "approved": bool, "restored": "<u128 decimal>" }`

Errors: `-32061` with registry error message.

boing_qaPoolList

List pending items in the community QA pool (same `tx_hash` keys as `-32051`).

Field	Type	Description
Params	<code>[]</code>	None
Result	<code>{ items: [...] }</code>	Each item: <code>tx_hash</code> , <code>bytecode_hash</code> , <code>deployer</code> (hex), <code>allow_votes</code> , <code>reject_votes</code> , <code>age_secs</code> , and when present on the signed deploy: <code>purpose_category</code> , <code>asset_name</code> , <code>asset_symbol</code> .

boing_qaPoolVote

Cast a vote on a pending pool item. When quorum and allow/reject thresholds are met, the item is resolved; on **Allow**, the stored signed transaction is inserted into the mempool.

Field	Type	Description
Params	<code>[tx_hash_hex, voter_hex, vote]</code>	<code>vote</code> is <code>allow</code> , <code>reject</code> , or <code>abstain</code> (case-insensitive). Only accounts listed in governance <code>qa_pool_config.administrators</code> may vote, unless <code>dev_open_voting</code> is true with an empty admin list (local dev).
Result	<code>{ outcome: "pending" "reject" "allow", mempool?: boolean, duplicate?: boolean, error?: string }</code>	On <code>allow</code> with <code>mempool: true</code> , the tx is in the mempool.

Errors: `-32052` no pending item for `tx_hash`; `-32053` voter is not a governance QA administrator; `-32057` operator authentication required (see **Operator RPC** above).

boing_operatorApplyQaPolicy

Replace the node's in-memory QA registry and pool governance config (same effect as loading `qa_registry.json` / `qa_pool_config.json` at startup, plus persistence to the node data directory). Intended for operators; requires **X-Boing-Operator** when `BOING_OPERATOR_RPC_TOKEN` is set.

Field	Type	Description
Params	<code>[qa_registry_json, qa_pool_config_json]</code>	Two strings , each the full JSON document (not a path).
Result	<code>{ ok: true }</code>	Policy applied.

Errors: `-32057` missing or wrong operator header when the token is configured; `-32602` invalid JSON or schema.

CLI: `boing qa apply --registry <path> --pool <path> [--operator-token ...]` (also reads `BOING_OPERATOR_RPC_TOKEN`).

boing_qaPoolConfig

Read effective QA pool governance parameters and current queue depth (no params).

Field	Type	Description
Params	<code>[]</code>	None
Result	object	<code>max_pending_items</code> , <code>max_pending_per_deployer</code> , <code>review_window_secs</code> , <code>quorum/threshold</code> fractions, <code>default_on_expiry</code> , <code>dev_open_voting</code> , <code>administrator_count</code> , <code>accepts_new_pending</code> , <code>pending_count</code> .

boing_getQaRegistry

Return the **effective protocol QA rule registry** the node uses for deployment checks (read-only, no authentication). Same JSON shape as on-disk `qa_registry.json` and as the first argument to `boing_operatorApplyQaPolicy`.

Field	Type	Description
Params	<code>[]</code>	None
Result	object	<code>max_bytecode_size</code> , <code>blocklist</code> (array of 32-byte arrays), <code>scam_patterns</code> , <code>always_review_categories</code> (array of strings), <code>content_blocklist</code> .

Reference: Canonical baseline JSON for comparison lives in the monorepo at `docs/config/qa_registry.canonical.json` — see `docs/config/CANONICAL-QA-REGISTRY.md`. Live nodes may differ after governance updates.

boing_clientVersion

Field	Type	Description
Params	<code>[]</code>	None

Result: string — e.g. `boing-node/0.1.0` (crate version). Lets clients and `npm run probe-rpc` confirm they are talking to a freshly built binary.

boing_rpcSupportedMethods

Field	Type	Description
Params	<code>[]</code>	None

Result: JSON array of strings — sorted list of `boing_*` method names implemented by this process (including `boing_clientVersion` and `boing_rpcSupportedMethods`). Intended for discovery and debugging; keep in sync with the node’s RPC router when adding methods.

boing_chainHeight

Return the height of the latest **committed** block on this node (see [Chain tip, committed height, and finality](#)).

Field	Type	Description
Params	<code>[]</code>	None

Result: `u64`

boing_health

Lightweight **liveness** and **build identity** for probes, dashboards, and load balancers. Single round-trip; includes committed `head_height` and the same optional `chain_id` / `chain_name` semantics as `boing_getNetworkInfo` (from `BOING_CHAIN_ID` / `BOING_CHAIN_NAME` on the node process).

Field	Type	Description
Params	<code>[]</code>	None

Result:

```
{
  "ok": true,
  "client_version": "boing-node/0.1.0",
  "chain_id": 6913,
  "chain_name": "Boing Testnet",
  "head_height": 42,
  "rpc_surface": {
    "jsonrpc_batch_max": 32,
    "websocket_max_connections": 0,
    "http_rate_limit_requests_per_sec": 100,
    "ready_min_peers": null,
    "http_max_body_megabytes": 8,
    "get_logs_max_block_range": 128,
    "get_logs_max_results": 2048,
    "max_log_topic_filters": 4
  },
  "rpc_metrics": {
    "rate_limited_total": 0,
    "json_parse_errors_total": 0,
    "batch_too_large_total": 0,
    "method_not_found_total": 0,
    "websocket_cap_rejects_total": 0
  }
}
```

- **chain_id / chain_name** : `null` when the corresponding env var is unset or invalid (same as `boing_getNetworkInfo`).
- **rpc_surface** : Operator-facing snapshot of limits (same sources as env vars `BOING_RPC_MAX_BATCH` , `BOING_RPC_WS_MAX_CONNECTIONS` , `BOING_RPC_MAX_BODY_MB` , node `RateLimitConfig.requests_per_sec` , `BOING_RPC_READY_MIN_PEERS` , plus fixed `boing_getLogs` bounds). `ready_min_peers` is `null` when no minimum is configured.
- **rpc_metrics** : Cumulative counters since process start (best-effort diagnostics).

boing_getSyncState

Structured view of the node’s committed chain tip for clients that want explicit **head** vs **finalized** fields. Today both heights are identical.

Field	Type	Description
Params	<code>[]</code>	None

Result:

```
{
  "head_height": 42,
  "finalized_height": 42,
  "latest_block_hash": "0x..."
}
```

- **head_height** : Same value as `boing_chainHeight` .
- **finalized_height** : Same as `head_height` in the current implementation; may diverge in a future release if the node exposes pre-commit or optimistic data.
- **latest_block_hash** : BLAKE3 hash of the tip block’s header (`Block::hash()`), 32-byte hex with `0x` prefix.

boing_getNetworkInfo

Single-call snapshot for dApps: chain metadata (from env when configured), tip state, timing, client build, consensus summary, `chain_native` aggregates, and `rpc.not_available` (see [dApp network discovery](#)).

Field	Type	Description
Params	<code>[]</code>	None

Result (shape):

```

{
  "chain_id": 6913,
  "chain_name": "Boing Testnet",
  "head_height": 42,
  "finalized_height": 42,
  "latest_block_hash": "0x...",
  "target_block_time_secs": 2,
  "client_version": "boing-node/0.1.0",
  "consensus": {
    "validator_count": 4,
    "model": "hotstuff_bft"
  },
  "native_currency": {
    "symbol": "BOING",
    "decimals": 18
  },
  "chain_native": {
    "account_count": 42,
    "total_balance": "...",
    "total_stake": "...",
    "total_native_held": "...",
    "as_of_height": 42
  },
  "developer": {
    "repository_url": "https://github.com/Boing-Network/boing.network",
    "rpc_spec_url": "https://github.com/Boing-Network/boing.network/blob/main/docs/RPC-API-SPEC.md",
    "dapp_integration_doc_url": "https://github.com/Boing-Network/boing.network/blob/main/docs/BOING-DAPP-INTEG",
    "sdk_npm_package": "boing-sdk",
    "websocket": {
      "path": "/ws",
      "handshake": { "type": "subscribe", "channel": "newHeads" },
      "event_types": ["newHead"]
    },
    "api_discovery_methods": [
      "boing_getRpcMethodCatalog",
      "boing_getRpcOpenApi",
      "boing_rpcSupportedMethods"
    ],
    "dex_discovery_methods": [
      "boing_listDexPools",
      "boing_listDexTokens",
      "boing_getDexToken"
    ],
    "http": {
      "live_path": "/live",
      "ready_path": "/ready",
      "jsonrpc_post_path": "/",
      "response_header_rpc_version": "x-boing-rpc-version"
    }
  },
  "end_user": {
    "chain_display_name": "Boing Testnet",
    "explorer_url": null,
    "faucet_url": null,
    "canonical_native_cp_pool": "0x7247ddc3180fdc4d3fd1e716229bfa16bad334a07d28aa9fda9ad1bfa7bdacc3",
    "canonical_native_dex_factory": "0x58112627fc84618a27b82e9af82bc9a51761c6d3cca1260c93d56d22b6c481a1",

```

```

"canonical_native_dex_multihop_swap_router": "0xf801cd1aa5ec402f89a2f394b49e6b0c136264d8945b16a4a6a81a188b1
"canonical_native_dex_ledger_router_v2": "0x33334ff73c44c93335ac5e69938a52ea65fa77b062d1961ed22c131adaa31e0
"canonical_native_dex_ledger_router_v3": "0x2c90ffcdddeb2683219b4b8143a91d7b93f249bcb0d9523c8b4f2111de668b79
"canonical_native_amm_lp_vault": "0x937d09ee8e4dcc521c812566ad4930792e74ad004ecb3ae2cc73dc015813aa8d",
"canonical_native_lp_share_token": "0x101201403f573e5b1d6d5c6b93d52d12c68957f4a228d5dad76e78c747044421"
},
"rpc": {
  "not_available": ["staking_apy"],
  "not_available_note": "..."
}
}

```

- **developer.dex_discovery_methods** : Stable list of L1 native DEX discovery JSON-RPC method names (same triple as `docs/HANDOFF_Boing_Network_Global_Token_Discovery.md`). Clients may merge this with `boing_rpcSupportedMethods` / `boing_getRpcMethodCatalog` for capability detection.
- **end_user** : Wallet-facing strings and optional **32-byte hex** hints when operators set the matching `BOING_CANONICAL_NATIVE_*` env vars on the node: `canonical_native_cp_pool`, `canonical_native_dex_factory`, `canonical_native_dex_multihop_swap_router`, `canonical_native_dex_ledger_router_v2`, `canonical_native_dex_ledger_router_v3`, `canonical_native_amm_lp_vault`, `canonical_native_lp_share_token`. Omitted keys are JSON `null`. `boing-sdk mergeNativeDexIntegrationDefaults` merges these with app overrides; historical embedded **6913** constants apply **only** when network info is omitted (offline). A live snapshot with null canonical fields means the hosted chain has not published those contracts ([BOING-DAPP-INTEGRATION.md](#), [dApp network discovery](#)).
- **chain_id / chain_name** : From `BOING_CHAIN_ID` / `BOING_CHAIN_NAME` when set on the node process; `chain_id` is `null` when not configured.
- **validator_count** : Size of this node's in-process validator set (HotStuff BFT); not a network-wide discovery endpoint for all operators.
- **native_currency.decimals** : Display hint for integrators; balances on `boing_getBalance` / `boing_getAccount` are still whole-unit u128 strings per those methods.
- **chain_native** : Sums over this node's committed account table; see [dApp network discovery](#).
- **developer** : Stable links and discovery hints for integrators. URLs default to the public GitHub tree; operators may override with environment variables (see table below).

Variable	Effect
<code>BOING_DEVELOPER_REPOSITORY_URL</code>	Base repository URL used to derive default doc links when spec/dapp URLs are unset.
<code>BOING_DEVELOPER_RPC_SPEC_URL</code>	Overrides <code>developer.rpc_spec_url</code> .
<code>BOING_DEVELOPER_DAPP_DOC_URL</code>	Overrides <code>developer.dapp_integration_doc_url</code> .

HTTP probes GET /live and GET /ready

Plain-text endpoints for orchestrators (Kubernetes `livenessProbe` / `readinessProbe`, Docker `HEALTHCHECK`, load balancers) without parsing JSON-RPC.

Path	Meaning
GET /live	Process is running and the HTTP server is accepting connections. Does not read chain state.
GET /ready	Node can acquire a read lock on in-memory state (RPC is not wedged on the state mutex). Optionally also requires a minimum P2P peer count when <code>BOING_RPC_READY_MIN_PEERS</code> is set (see below).

Successful responses use HTTP **200** with a short body (`ok` / `ready`). If the peer requirement is not met, `GET /ready` returns **503** with a short `not_ready: peers ...` message. They receive the same `x-boing-rpc-version` response header as `POST /`.

Request body limit (JSON-RPC POST /)

Large signed-transaction hex payloads (e.g. contract deploy) need a generous HTTP body limit. Default maximum request body size is **8 MiB**.

Variable	Effect
<code>BOING_RPC_MAX_BODY_MB</code>	Maximum JSON-RPC POST body size in mebibytes (integer, ≥ 1). Invalid or missing values fall back to 8 .
<code>BOING_RPC_MAX_BATCH</code>	Max objects per JSON-RPC batch array (default 32 , hard cap 256). 0 is invalid and falls back to the default.
<code>BOING_RPC_READY_MIN_PEERS</code>	When set to a positive integer, <code>GET /ready</code> returns 503 unless the node reports at least that many connected P2P peers. Unset or 0 : no peer check (read lock only).
<code>BOING_RPC_WS_MAX_CONNECTIONS</code>	Max concurrent <code>GET /ws</code> WebSocket connections (0 = unlimited). When exceeded, new handshakes receive 503 .

WebSocket GET /ws — newHeads

Browser-friendly push when the **local** committed tip changes (block produced or imported). Uses the same CORS allowlist as `POST /` plus `BOING_RPC_CORS_EXTRA_ORIGINS`.

1. Connect with `GET /ws` (WebSocket upgrade).

2. Send **one** text frame: `{"type":"subscribe","channel":"newHeads"}` .
3. Server replies `{"type":"subscribed","channel":"newHeads"}` .
4. On each committed tip update, server sends `{"type":"newHead","height":<u64>,"hash":"0x..."}` (BLAKE3 header hash, 32-byte hex).

Laggy subscribers may drop intermediate heads (**broadcast** channel); clients should reconcile with `boing_getSyncState` / `boing_chainHeight` if they need every height.

Public RPC operators may set `BOING_RPC_WS_MAX_CONNECTIONS` so `newHeads` cannot exhaust file descriptors or CPU from unbounded subscribers.

boing_getRpcMethodCatalog

Returns the embedded **method catalog** (JSON Schema-style `params` / `result` hints per method) shipped with the node binary. Intended for codegen, docs generators, and IDE tooling.

Field	Type	Description
Params	<code>[]</code>	None

Result: JSON object with `description` and `methods` (array of `{ name, summary, params, result }`).

boing_getRpcOpenApi

Returns a **minimal OpenAPI 3.1** JSON document describing `POST /` (JSON-RPC envelope), `GET /ws` (subscribe protocol), and plain-text probes `GET /live` / `GET /ready` . Complements the human-readable spec and `boing_getRpcMethodCatalog` .

Field	Type	Description
Params	<code>[]</code>	None

Result: OpenAPI root object (`openapi` , `info` , `paths` , `components` , ...).

boing_getBalance

Get the spendable balance for an account. **Recommended for wallets** (e.g. boing.express) to display balance without deriving from state.

Field	Type	Description
Params	[hex_account_id]	32-byte AccountId (hex)
Result	{ balance: string }	Native BOING balance as whole units (u128 decimal string). Wallets and explorers should not assume a fixed 10^{18} “wei”-style divisor unless a future token standard adds it.

Example: {"jsonrpc": "2.0", "id": 1, "method": "boing_getBalance", "params": ["0x..."]} → {"jsonrpc": "2.0", "id": 1, "result": {"balance": "1000000"}}

boing_getAccount

Get full account state (balance, nonce, stake). **Recommended for wallets** to build transactions (nonce) and show balance/stake.

Field	Type	Description
Params	[hex_account_id]	32-byte AccountId (hex)
Result	{ balance: string, nonce: number, stake: string, pending_unbond: string, unbond_unlock_height: number }	balance, stake, and pending_unbond are u128 as decimal strings; nonce and unbond_unlock_height are u64. If account does not exist, returns zeros. pending_unbond is stake waiting for the unbonding delay; claim with ClaimUnbond after unbond_unlock_height.

Example: {"jsonrpc": "2.0", "id": 1, "method": "boing_getAccount", "params": ["0x..."]} → {"jsonrpc": "2.0", "id": 1, "result": {"balance": "1000000", "nonce": 5, "stake": "0", "pending_unbond": "0", "unbond_unlock_height": 0}}

boing_getBlockByHeight

Get a block by height.

Field	Type	Description
Params	[height] or [height, include_receipts]	Block height (u64); optional boolean — when true, adds receipts array (same order as transactions, null if no receipt cached).

Result: Block object or null if not found. header includes receipts_root (hex-encoded 32-byte Merkle root over execution receipts; see protocol spec). Optional hash field (hex) is added for convenience. With include_receipts: true, each entry matches boing_getTransactionReceipt shape or null.

Off-chain deploy registry: indexers that need a **chain-wide list of predicted `ContractDeploy*` contract addresses** (and BLAKE3 **transaction ids**) can call this method with `include_receipts: false` and parse `transactions` . The TypeScript SDK exposes `extractUniversalContractDeploymentsFromBlock` and related helpers (`boing-sdk` , `universalContractDeployIndex.ts`). An optional Cloudflare Worker materializes rows into D1 and serves HTTP/SSE; see [HANDOFF_Universal_Contract_Deploy_Indexer.md](#).

boing_getTransactionReceipt

Lookup execution outcome for an included transaction (same id as `Transaction::id()` / `mempool tx_hash`).

Field	Type	Description
Params	<code>[hex_tx_id]</code>	32-byte transaction id (hex)

Result: Receipt object or `null` if unknown (e.g. not yet included, or node predates receipt persistence).

Receipt: `{ tx_id, block_height, tx_index, success, gas_used, return_data, logs, error }` — `return_data` is hex; `logs` is an array of `{ topics: string[], data: string }` (each topic is `0x` + 32-byte hex, `data` is hex). Logs are populated for `ContractCall` and for **ContractDeploy only when bytecode uses init-code mode** (leading `0xFD` prefix; see [TECHNICAL-SPECIFICATION.md](#) §4.4). `error` is set when `success` is false.

boing_getLogs

Query execution logs across a **bounded** range of committed blocks. Intended for indexers and wallets; nodes enforce limits to keep scans cheap.

Field	Type	Description
Params	[filter]	Single object (camelCase keys below).
fromBlock	number or string	Start height (inclusive). JSON number or decimal / 0x hex string.
toBlock	number or string	End height (inclusive). Must be <code>>= fromBlock</code> .
address	string (optional)	32-byte <code>AccountId</code> hex. When set, only logs attributed to that contract are returned (<code>ContractCall</code> target, or the deployed contract address for deploy txs that used init-code mode — leading bytecode byte <code>0xFD</code> , see <code>TECHNICAL-SPECIFICATION.md §4.4</code>).
topics	array (optional)	Up to 4 entries; each is <code>null</code> (wildcard) or a 32-byte topic hex string. Positional match: entry <code>i</code> filters <code>topic_i</code> of each log (same index slot).

Limits (reference implementation): inclusive block span at most **128**; at most **2048** log entries per call. Exceeding span returns JSON-RPC error `-32602`; exceeding the result cap returns `-32603`.

Result: JSON array of objects:

```
{ block_height, tx_index, tx_id, log_index, address, topics, data }
```

- `address` is hex or `null` when the node cannot attribute an emitting contract.
- Order is block height ascending, then transaction order in the block, then `log_index`.

Example: `{"jsonrpc": "2.0", "id": 1, "method": "boing_getLogs", "params": [{"fromBlock": 1, "toBlock": 10, "topics": [null, "0x000..."]}]}`

boing_getBlockByHash

Get a block by hash.

Field	Type	Description
Params	[hex_block_hash] or [hex_block_hash, include_receipts]	Optional boolean — same as <code>boing_getBlockByHeight</code> .

Result: Block object or `null` if not found.

boing_getAccountProof

Get a Merkle proof for an account.

Field	Type	Description
Params	[hex_account_id]	32-byte AccountId (hex)

Result: { proof: string, root: string, value_hash: string }

boing_verifyAccountProof

Verify an account Merkle proof.

Field	Type	Description
Params	[hex_proof, hex_state_root]	Proof and expected root (hex)

Result: { valid: boolean }

boing_getContractStorage

Read a single 32-byte Boing VM storage word for a contract (same semantics as `SLOAD`: missing slot → zero word). Useful for indexers and wallets that know the storage key layout (e.g. reference NFT / token conventions).

Field	Type	Description
Params	[hex_contract_id, hex_storage_key]	Two 32-byte values (hex)

Result: { value: string } — value is 0x + 64 hex chars (32 bytes).

boing_listDexPools / boing_listDexTokens / boing_getDexToken

First-class **DEX-derived discovery** (see [HANDOFF_Boing_Network_Global_Token_Discovery.md](#)): pools and tokens come from the on-chain pair directory plus live pool storage reads — **not** “every ledger account.”

Field	Type	Description
Env	—	BOING_CANONICAL_NATIVE_DEX_FACTORY (same id as boing_getNetworkInfo.end_user.canonical_native_dex_factory) when params.factory is omitted. If neither is set, these methods return -32000 . Optional BOING_DEX_TOKEN_METADATA_SCAN_BLOCKS (default 8192 , max 500000): how many recent blocks the node scans for ContractDeployWithPurposeAndMetadata (asset_symbol / asset_name) when enriching token rows. Optional BOING_DEX_DISCOVERY_MAX_RECEIPT_SCANS : max receipt rows examined per call for Log3 register matching (default 500000 ; set 0 for unlimited). Optional BOING_DEX_TOKEN_DECIMALS_JSON : JSON object { "0x" + 64hex: number } overriding default decimals (18) for boing_listDexPools (tokenADecimal / tokenBDecimals), boing_listDexTokens , and boing_getDexToken .
Params	[object]	Single JSON object (or [object] one-element array). Common fields: factory (optional 32-byte hex directory id — overrides env for multi-directory / self-serve RPC), cursor , limit . light: true (or enrich: false) skips receipt + deploy scans: createdAtHeight / firstSeenHeight are JSON null , and token metadataSource is abbrev only. Token listing also accepts minReserveProduct and minLiquidityWei as decimal digit strings (unsigned integers). includeDiagnostics: true adds a top-level diagnostics object (pool and token responses) or embeds it on boing_getDexToken when present.

Persistence: When the node is started with a **data directory** and receipt persistence enabled, committed **ExecutionReceipt** blobs (**receipts_{height}.bin**) for heights **0..=tip** are loaded into the in-memory receipt map used by these methods, so **createdAtHeight** / **firstSeenHeight** reflect every receipt the process has loaded (same trust model as **boing_getTransactionReceipt** for those heights).

boing_listDexPools result: **{ pools: [...], nextCursor: string | null }** — each pool has **poolHex**, **tokenAHex**, **tokenBHex**, **tokenADecimal** / **tokenBDecimals** (same resolution as token-list **decimals**: **BOING_DEX_TOKEN_DECIMALS_JSON**, default **18**), **feeBps**, **reserveA** / **reserveB** (decimal strings), and **createdAtHeight**: committed block height of a validated factory **Log3** register for that pool when known (else JSON **null**, e.g. **light** mode). Optional **diagnostics** when **includeDiagnostics** was set.

boing_listDexTokens result: **{ tokens: [...], nextCursor: string | null }** — each token has **id**, **symbol**, **name**, **decimals**, **poolCount**, **firstSeenHeight** (minimum known pool registration height for that token, else **null**), and **metadataSource**: **deploy** when **symbol** / **name** were taken from a recent metadata deploy, else **abbrev**. **nextCursor** is the last token's **id** when another page may exist; otherwise **null**. Optional **diagnostics** when **includeDiagnostics** was set.

boing_getDexToken params: **{ "id": "0x" + 64 hex, "factory"?: ..., "light"?: bool, "includeDiagnostics"?: bool }** — **result** is one token object or JSON **null**. **diagnostics** may appear on the token object when requested.

Pagination cursors: pool pages use opaque strings of the form `i{start_index}` (implementation detail; treat as opaque). Token pages use the last returned `id` as the next `cursor`.

SDK: `BoingClient.listDexPoolsPage`, `listDexTokensPage`, `getDexToken`.

Regression: `cargo test -p boing-node --test native_dex_factory_rpc_happy_path`.

Native constant-product AMM (chain 6913 — integration note)

Wallets and dApps still **configure** the pool `AccountId` in their own env (the JSON-RPC methods are generic). For **public testnet**, operators maintain a single **canonical** pool id so tutorials and UIs can default to one known-good deployment.

Canonical public testnet pool <code>AccountId</code>	Hosted Fly RPC currently unset (<code>end_user.canonical_native_cp_pool</code> is <code>null</code> as of 2026-08-24). Historical tunnel-ledger id <code>0x7247ddc3180fdc4d3fd1e716229bfa16bad334a07d28aa9fda9ad1bfa7bdacc3</code> (operator-published 2026-05-21 , documented deployer <code>0x3b6a27bcceb6a42d62a3a8d02a6f0d73653215771de243a63ac048a18b59da29</code>) is not live on this cluster. Read <code>boing_getNetworkInfo.end_user</code> . After bootstrap, follow OPS-CANONICAL-TESTNET-NATIVE-AMM-POOL.md . CREATE2 note: deployer <code>0xc063512f...</code> + current-tree bytecode predicts <code>0xce4f8193...</code> — not the live pool; run <code>cargo run -p boing-execution --example verify_canonical_cp_pool_create2_drift</code> .
boing.finance (separate repo)	Source constant: <code>frontend/src/config/boingCanonicalTestnetPool.js</code> → <code>CANONICAL_BOING_TESTNET_NATIVE_CP_POOL_HEX</code> . Cloudflare Pages / CI build: <code>REACT_APP_BOING_NATIVE_AMM_POOL</code> (same 64-hex <code>AccountId</code>). Also wire <code>nativeConstantProductPool</code> (or equivalent in <code>contracts.js</code>) for chain 6913 . Redeploy the app after any change to this hex. Optional: depend on npm <code>boing-sdk</code> and import <code>CANONICAL_BOING_TESTNET_NATIVE_CP_POOL_HEX</code> (mirror of this table — see row below) plus shared RPC helpers / <code>explainBoingRpcError</code> .
boing-sdk	<code>CANONICAL_BOING_TESTNET_NATIVE_CP_POOL_HEX</code> — exported from <code>boing-sdk/src/canonicalTestnet.ts</code> ; convenience for tutorials and TS apps. Authoritative values remain this spec and TESTNET.md §5.3.

Need	RPC / approach
Reserve A / B	Two <code>boing_getContractStorage</code> calls: <code>hex_contract_id</code> = pool account; storage keys are the 32-byte big-endian layout for the pool program (see NATIVE-AMM-CALLDATA.md and <code>boing_execution::reserve_a_key / reserve_b_key</code>). Each <code>value</code> word holds the u128 reserve in the low 16 bytes (high 16 zero), matching reference-token word style.
Swap / liquidity	Signed <code>contract_call</code> transactions (calldata per NATIVE-AMM-CALLDATA.md); preflight <code>boing_simulateTransaction</code> / <code>boing_qaCheck</code> on deploy bytecode as usual. Canonical pool deploy : prefer <code>create2_salt: Some(NATIVE_CP_POOL_CREATE2_SALT_V1)</code> on purpose deploy (fixed salt in <code>native_amm.rs</code>) so the pool <code>AccountId</code> is predictable from deployer + bytecode — see NATIVE-AMM-CALLDATA.md § CREATE2.
Automated regression	Repository test: <code>cargo test -p boing-node --test native_amm_rpc_happy_path</code> (deploy → add liquidity → swap; asserts reserves via <code>boing_getContractStorage</code>).

Publish checklist (for **future** pool rotations): [OPS-CANONICAL-TESTNET-NATIVE-AMM-POOL.md](#). Keep **boing.finance** and **boing-sdk** mirrors in sync with live **end_user** when a canonical pool is published.

Native DEX directory + ledger router (integration note)

Multi-pair flows use **generic** JSON-RPC (`contract_call` , `getContractStorage` , `getLogs` , `simulateTransaction`). Nodes with **BOING-CANONICAL-NATIVE-DEX-FACTORY** also expose **boing_listDexPools** / **boing_listDexTokens** / **boing_getDexToken** for cursor-paginated discovery (see above). Specs and bytecode live in the node repo:

Artifact	Doc	Notes
Pair directory	NATIVE-DEX-FACTORY.md	<code>register_pair</code> / <code>pairs_count</code> / <code>get_pair_at</code> ; count key + <code>Log3</code> indexing
Ledger router	NATIVE-DEX-LEDGER-ROUTER.md	Forwards 128-byte pool calldata via nested <code>Call</code> — v1/v3 pools only
SDK	<code>boing-sdk</code>	<code>nativeDexFactory*</code> , <code>nativeDexLedgerRouter</code> , <code>create2</code> prediction helpers
Regression	<code>cargo test -p boing-node --test native_dex_factory_rpc_happy_path</code>	Deploy pool + directory → <code>register_pair</code> → <code>pairs_count</code> / <code>get_pair_at</code> receipts

Materialized pool directory (HTTP, not JSON-RPC): this monorepo ships a Cloudflare Worker ([workers/native-dex-indexer/](#)) that mirrors the indexer's `pools[]` into **D1** and exposes `GET /v1/directory/meta` and cursor-paginated `GET /v1/directory/pools` . Use **boing-sdk**

`fetchNativeDexDirectoryMeta` , `fetchNativeDexDirectoryPoolsPage` ,
`collectAllNativeDexDirectoryPools` (`nativeDexDirectoryApi.ts`). Ops, secrets, R2, and protocol follow-ups: [HANDOFF_NATIVE_DEX_DIRECTORY_R2_AND_CHAIN.md](#). Local smoke: `npm run verify-native-dex-directory-worker` from the repo root.

boing_simulateContractCall

Dry-run a `contract_call` without building or signing a full `SignedTransaction` . Execution uses the same VM path as `boing_simulateTransaction` on a **state snapshot** (no commit to committed chain).

Field	Type	Description
Params	[<code>contract_hex</code> , <code>calldata_hex</code> , <code>sender_hex?</code> , <code>at_block?</code>]	<code>contract_hex</code> : 32-byte <code>AccountId</code> (<code>0x</code> + 64 hex). <code>calldata_hex</code> : VM calldata (hex). Max 256 KiB decoded length. <code>sender_hex</code> : optional; JSON <code>null</code> or omit → synthetic all-zero <code>AccountId</code> (ephemeral account with nonce 0 inserted in the snapshot if missing). Otherwise <code>sender</code> must exist in committed state; nonce is taken from that account’s committed nonce. <code>at_block</code> : optional <code>"latest"</code> , <code>null</code> , or integer equal to current committed tip height ; other heights return <code>-32602</code> (historical replay not implemented).
Result	object	Same shape as <code>boing_simulateTransaction</code> <code>result</code> : <code>gas_used</code> , <code>success</code> , <code>return_data</code> , <code>logs</code> , <code>error</code> (when <code>success</code> is false), <code>suggested_access_list</code> , <code>access_list_covers_suggestion</code> . The synthetic tx uses <code>access_list</code> = heuristic suggestion for <code>ContractCall</code> , so <code>access_list_covers_suggestion</code> is <code>true</code> .

Errors: `-32602` for invalid params (malformed hex, calldata too long, unknown `at_block` , missing sender account when non-zero sender). VM execution failures return `success: false` in `result` (same as `boing_simulateTransaction`), not a JSON-RPC error.

Tests: `cargo test -p boing-node --test simulate_contract_call_rpc` .

Extended rationale and ecosystem rollout: [PROTOCOL_NATIVE_DEX_RPC_AND_INDEXING_ROADMAP.md](#) §1.

boing_simulateTransaction

Simulate a transaction without applying it.

Field	Type	Description
Params	[<code>hex_signed_tx</code>]	Hex-encoded <code>SignedTransaction</code>

Result: { gas_used: number, success: boolean, return_data: string, logs?: array, error?: string, suggested_access_list: { read: string[], write: string[] }, access_list_covers_suggestion: boolean } — on success, `return_data` is hex (contract return buffer) and `logs` matches receipt log shape; on failure, `return_data` is "0x" and `logs` is []. **suggested_access_list** is a **heuristic** minimum account set for parallel scheduling from the tx payload (see `TECHNICAL-SPECIFICATION.md §4.2`); **access_list_covers_suggestion** is `true` when the signed tx's declared access list includes every account in that suggestion (read or write). Hints are included on both success and failure so clients can fix access lists before submit.

For **unsigned** calldata-only simulation, use `boing_simulateContractCall` above.

boing_registerDappMetrics

Register a dApp for incentive tracking.

Field	Type	Description
Params	[hex_contract, hex_owner]	Contract and owner AccountIds (hex)

Result: { registered: true, contract: string, owner: string }

boing_submitIntent

Submit a signed intent for solver fulfillment.

Field	Type	Description
Params	[hex_signed_intent]	Hex-encoded SignedIntent

Result: { intent_id: string }

boing_qaCheck (optional — when QA is enabled)

Pre-flight check for a deployment without submitting. Allows clients to see whether bytecode (and optional purpose declaration) would be **Allow**, **Reject**, or **Unsure** (pool) before calling `boing_submitTransaction`.

Field	Type	Description
Params	[hex_bytecode] or [hex_bytecode, purpose_category, description_hash?, asset_name?, asset_symbol?]	Bytecode only runs size/opcode/blocklist rules. With <code>purpose_category</code> , the same full mempool QA path applies as in <code>boing_submitTransaction</code> for deploy payloads (including optional 32-byte <code>description_hash</code> , then optional <code>asset_name</code> / <code>asset_symbol</code> for content policy). To pass only name/symbol without a real description commitment, use a placeholder 32-byte hex (e.g. all zeros).
Result	{ result, rule_id?, message?, doc_url? }	<code>result</code> : "allow", "reject" (rule_id/message when applicable), or "unsure" (community QA pool). Mirrors mempool <code>boing_qa</code> for contract deploy.

Errors: When QA is not enabled, returns `-32601` (method not found) or a dedicated code. When QA rejects: use structured error code below.

boing_faucetRequest (testnet only)

Request testnet BOING for an account. Only available when the node is started with `--faucet-enable`. **Do not enable on mainnet.**

Field	Type	Description
Params	[hex_account_id]	32-byte account ID (hex). Recipient of the faucet transfer.

Result: { ok: true, amount: number, to: string, message: string, tip_height: number, connected_peers: number, warning?: string }

- **tip_height** / **connected_peers**: Chain tip and P2P peer count at accept time.
- **warning**: Present when tip is **0** (especially with **no peers**) — the transfer is only in the **local mempool** and may **never commit** until the node syncs or a validator produces blocks. Clients **MUST NOT** treat `ok: true` alone as "balance credited."

Rate limit: 1 request per 60 seconds per account ID. Returns `-32016` with message "Faucet cooldown" if called too soon.

P2P: Same as `boing_submitTransaction` — when `--p2p-listen` is set and the built transfer is admitted, the signed faucet tx is **gossiped** on `boing/transactions`.

Errors: `-32601` Faucet not enabled; `-32000` Faucet account not initialized or balance too low.

Error Codes

Code	Meaning
-32600	Invalid Request
-32601	Method not found
-32602	Invalid params
-32000	Server error
-32016	Rate limit exceeded
-32050	QA: Deployment rejected — Transaction rejected by protocol QA (e.g. bytecode or purpose rule). Response SHOULD include <code>data: { rule_id: string, message: string }</code> for structured feedback. See QUALITY-ASSURANCE-NETWORK.md .
-32051	QA: Pending pool — Deployment referred to governance QA pool (result: Unsure). Response includes <code>data: { tx_hash: string }</code> (hex).
-32052	QA pool — No pending item for the given <code>tx_hash</code> .
-32053	QA pool — Voter is not a governance QA administrator.
-32054	QA pool disabled — Governance has not enabled the pool (e.g. no <code>administrators</code> and <code>dev_open_voting</code> false, or <code>max_pending_items</code> is 0).
-32055	QA pool full — Global <code>max_pending_items</code> reached; optional <code>data.reason: "pool_full"</code> .
-32056	QA pool deployer cap — Sender exceeded <code>max_pending_per_deployer</code> ; optional <code>data.reason: "deployer_cap"</code> .
-32057	Operator RPC auth — <code>boing_qaPoolVote</code> or <code>boing_operatorApplyQaPolicy</code> called without valid <code>X-Boing-Operator</code> while <code>BOING_OPERATOR_RPC_TOKEN</code> is set on the node.

Boing Network — Authentic. Decentralized. Optimal. Sustainable.